



ATEKA CLOUD

Ficha técnica del proyecto

Dato	Valor
Versión del documento	1.0
Fecha	Septiembre de 2026
Autor	Vlad
Repositorio principal	<code>cloud-core</code> → carpeta <code>akx-cloud-project/</code> (backend, web y app móvil)
Repositorio del gateway	<code>cloud-mid</code> (filtro de acceso del móvil y panel de control)

Cómo leer este documento

Los apartados **1 a 3** explican qué es la aplicación y cómo funciona por dentro. Con ellos tienes suficiente para orientarte.

Los apartados **4 a 7** son de consulta: tecnologías, carpetas, la API y la base de datos.

Los apartados **8 a 10** tratan de la seguridad, cómo arrancarla y qué limitaciones tiene.

Si una palabra no te suena (JWT, embedding, OCR...), búscala en el **Anexo A: Glosario**.

Índice

1. Qué es ATEKA Cloud	3
1.1 Qué puede hacer	3
1.2 Tipos de usuario	3
1.3 Las piezas del proyecto	4
2. Arquitectura	5
2.1 Qué es cada caja	5
3. Cómo funciona por dentro	6
3.1 Subir un archivo	6
3.2 Preguntar algo al asistente	6
3.3 Iniciar sesión y permisos	7
3.4 App móvil y gateway	7
3.5 Automatizaciones con n8n	8
4. Tecnologías	9
4.1 Backend	9
4.2 Frontend web	11
4.3 App móvil	11
4.4 Gateway y panel de control (cloud-mid)	12
5. Estructura de carpetas	13
5.1 Raíz del repositorio	13
5.2 Backend	13
5.3 Frontend web	17
5.4 App móvil	19
5.5 Gateway (repositorio cloud-mid)	20
6. La API: la conexión entre las apps y el servidor	22
6.1 Áreas de la API	22
6.2 Cómo se identifica cada cliente	22
7. Base de datos	24
7.1 PostgreSQL (ATEKA)	24
7.2 MariaDB (gateway cloud-mid)	25
8. Seguridad	26
9. Puesta en marcha y despliegue	27
9.1 Requisitos del servidor	27
9.2 Puertos	27
9.3 Variables del .env (las principales)	27
9.4 Comandos habituales	28
10. Limitaciones conocidas y pendientes	29
10.1 Limitaciones de diseño	29
10.2 Pendientes antes de publicar la app	29
Anexo A. Glosario	30
Anexo B. Librerías auxiliares	32

1. Qué es ATEKA Cloud

ATEKA Cloud es una aplicación para que las **empresas guarden sus documentos en la nube y trabajen con ellos con ayuda de inteligencia artificial**. Se parece a un Google Drive privado instalado en un servidor propio, con un asistente que entiende lo que hay **dentro** de los documentos.

Toda la inteligencia artificial funciona en **local** (con Ollama, en el propio servidor). Los documentos de los clientes no se envían a servicios externos como ChatGPT.

1.1 Qué puede hacer

- **Guardar y organizar archivos:** carpetas, arrastrar y soltar, mover, copiar, renombrar, papelera y descarga de carpetas en .zip.
- **Buscar por significado:** si buscas "contrato de alquiler", encuentra el documento aunque no se llame así, porque lee su contenido.
- **Asistente (chat):** se le pide en lenguaje normal ("mueve las facturas de mayo a la carpeta Contabilidad", "¿cuánto facturé en el primer trimestre?") y lo ejecuta.
- **Facturas automáticas:** al subir una factura, la aplicación la reconoce, extrae sus datos (emisor, cliente, importes, líneas), decide si es una **venta** o una **compra** y la incluye en informes.
- **Carpetas compartidas por rol:** por ejemplo, una carpeta "Contabilidad" que solo ve el rol "Contable".
- **Multiempresa:** una sola instalación da servicio a varias empresas, cada una con sus datos aislados.
- **App Android e integración con n8n** (automatizaciones; por ejemplo, un bot de Telegram que habla con el asistente).

1.2 Tipos de usuario

Tipo	Quién es	Qué puede hacer
Superadmin	El dueño de la plataforma. No pertenece a ninguna empresa.	Dar de alta, suspender y borrar empresas, y crear el primer administrador de cada una. Se crea solo al arrancar a partir del <code>.env</code> .
Admin	El administrador de una empresa cliente.	Crear miembros de su empresa, definir roles, crear carpetas compartidas y fijar el CIF de la empresa.
Miembro	Un empleado de la empresa.	Usar sus archivos y lo que le permitan sus roles.

No hay registro público: nadie puede crearse una cuenta por su cuenta. Las cuentas las crean el superadmin o el admin.

Además, cada admin crea **roles** con el nombre que quiera ("Contable", "Comercial"...) y les marca **capacidades** de una lista fija:

Capacidad	Qué desbloquea
<code>chat</code>	Usar el asistente. Sin ella, el chat ni siquiera aparece.
<code>facturas</code>	Escanear facturas y consultar su analítica (ventas, compras, IVA...).
<code>busqueda</code>	Búsqueda por significado y lectura del contenido de archivos desde el chat.
<code>gestion_archivos</code>	Crear, copiar, mover, renombrar y borrar archivos, y la papelera.

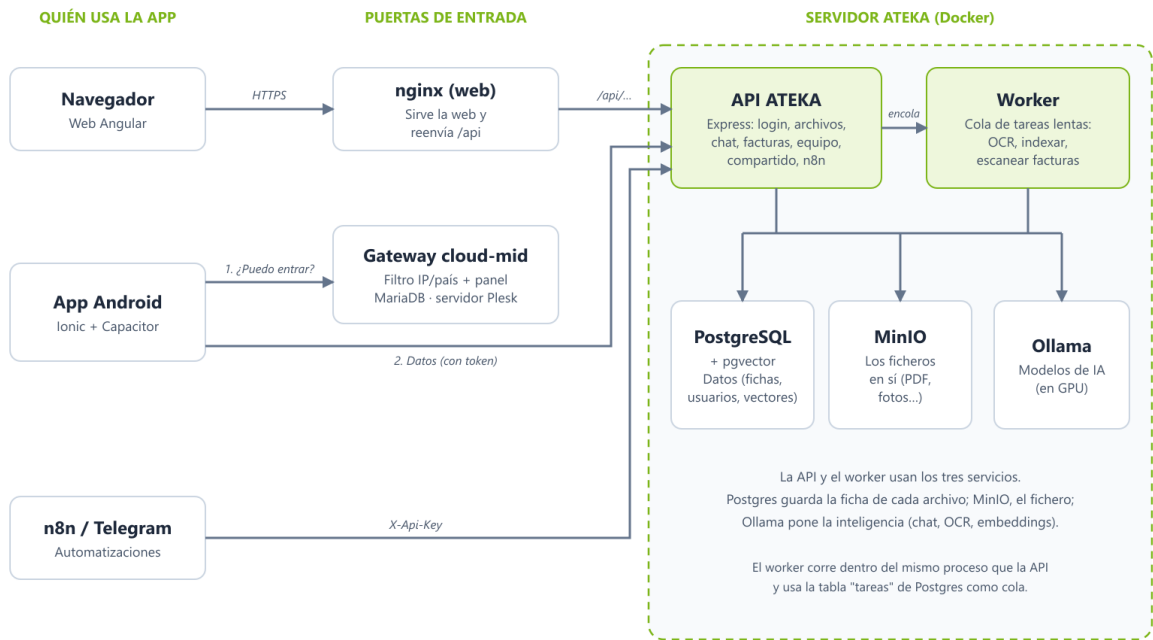
Los admins y el superadmin tienen todas las capacidades.

1.3 Las piezas del proyecto

Pieza	Dónde está	Para qué sirve
Backend (API)	<code>akx-cloud-project/backend/</code>	El servidor. Recibe las peticiones, guarda los datos y habla con la IA. Todo lo demás depende de él.
Frontend web	<code>akx-cloud-project/frontend/</code>	La página web que usa la gente desde el navegador.
App móvil	<code>akx-cloud-project/mobile/</code>	La app Android. Reutiliza la API del backend.
Gateway + panel	repositorio <code>cloud-mid</code>	Un "portero" para la app móvil: decide qué IP o país puede entrar. Incluye un panel de control para gestionarlo.
Infraestructura	<code>docker-compose.yml</code> (raíz)	Arranca todo con Docker: base de datos, almacenamiento, API y web.

2. Arquitectura

El diagrama muestra cómo se conectan las piezas. Se lee de izquierda a derecha: **quién usa la app, por dónde entra y qué hay en el servidor.**



Idea clave

Postgres sabe *qué* hay y *de quién* es. MinIO guarda *el fichero* en sí. Ollama *entiende* el contenido. La API coordina a los tres.

2.1 Qué es cada caja

Caja	Explicación sencilla
Navegador	La web en Angular. Descarga la página desde nginx y después pide los datos a la API.
nginx	Servidor web. Entrega los ficheros de la web (HTML, JS, CSS) y reenvía todo lo que empieza por <code>/api/</code> a la API. Así, para el navegador, web y API están en la misma dirección.
App Android	Antes de iniciar sesión pregunta al gateway si puede entrar (1). Si le dice que sí, habla con la API igual que la web (2).
Gateway cloud-mid	Portero con reglas de IP y país guardadas en su propia base de datos (MariaDB). Tiene un panel web para gestionar esas reglas y ver los dispositivos conectados.
n8n / Telegram	Automatizaciones externas. No usan usuario y contraseña, sino una clave API (cabecera <code>X-API-Key</code>).
API ATEKA	El corazón. Comprueba quién eres, qué puedes hacer y ejecuta la acción.
Worker	Un "ayudante en segundo plano". Hace las tareas lentas (leer un PDF, OCR de una foto, analizar una factura) sin que el usuario tenga que esperar.
PostgreSQL + pgvector	La base de datos: usuarios, empresas, fichas de archivos, facturas, cola de tareas y los vectores para buscar por significado.

Caja	Explicación sencilla
MinIO	El "disco duro" de la aplicación: guarda los bytes de cada fichero. Postgres solo guarda la ficha (nombre, dueño, carpeta...).
Ollama	Ejecuta los modelos de IA en la GPU: el del chat, el de embeddings y los de visión/OCR.

3. Cómo funciona por dentro

Cuatro recorridos típicos, paso a paso. La última columna indica el fichero donde está el código, para que sepas dónde mirar.

3.1 Subir un archivo

#	Qué pasa	Dónde
1	El usuario arrastra el archivo. La web comprueba antes la extensión (PDF, DOCX, XLSX, TXT, CSV, JPG, PNG, WEBP) para no subir algo que se va a rechazar.	<code>frontend/.../shared/tipos-archivo.ts</code>
2	La petición llega a <code>POST /api/archivos/subir</code> con el token. multer recibe el fichero en memoria (máximo 50 MB).	<code>archivos.routes.ts</code>
3	Se comprueba el tipo real del fichero mirando sus primeros bytes (<i>magic bytes</i>), no solo la extensión. Se calcula su huella SHA-256 : si ya existe uno idéntico, se reutiliza y no se procesa dos veces.	<code>archivos.controller.ts</code>
4	El fichero va a MinIO y su ficha a la tabla <code>archivos</code> de Postgres .	<code>archivos.service.ts</code>
5	Se añaden dos tareas a la cola (tabla <code>tareas</code>): indexar y autoescanear . El usuario ya tiene su respuesta; el resto ocurre en segundo plano.	<code>tareas.service.ts</code>
6	El worker extrae el texto: capa de texto del PDF, contenido del Word u OCR si es una imagen.	<code>extraccion.service.ts</code>
7	Ese texto se trocea (1 000 caracteres con 150 de solape), se convierte en vectores con bge-m3 y se guarda en la tabla <code>fragmentos</code> . A partir de aquí se puede encontrar por significado.	<code>rag.service.ts</code>
8	Si el texto parece una factura, la IA extrae sus datos, se corrigen emisor y cliente, se clasifica como venta o compra y se guarda.	<code>facturas.service.ts</code>
9	La columna Estado del explorador se actualiza sola (consulta cada 3 segundos).	<code>explorador.ts</code>

3.2 Preguntar algo al asistente

#	Qué pasa	Dónde
1	La web envía la conversación a <code>POST /api/chat</code> .	<code>chat.service.ts</code> (front)
2	Se comprueba que el rol tenga la capacidad <code>chat</code> . Si no la tiene, se responde 403 (prohibido).	<code>chat.service.ts</code>

#	Qué pasa	Dónde
3	Pre-flights: las frases habituales ("lista mis facturas de marzo", "vacía la papelera", "¿cuánto IVA pago este trimestre?") se reconocen con reglas fijas y se responden directamente desde la base de datos , sin usar la IA. Es más rápido y más fiable.	<code>chat.deteccion.ts</code> <code>chat.service.ts</code>
4	Si no encaja con ninguna regla, se envía solo el último mensaje al modelo junto con las herramientas que el rol puede usar. El modelo pide herramientas ("buscar", "mover"...) hasta resolverlo, con un máximo de 15 rondas.	<code>chat.tools.ts</code>
5	Si la acción es masiva e irreversible (por ejemplo, vaciar la papelera), primero se pide confirmación. La pregunta pendiente se guarda en la tabla <code>chat_pendientes</code> .	<code>chatPendientes.service.ts</code>
6	Se devuelve la respuesta en markdown, con tablas paginadas y botones "Abrir" si hay archivos concretos.	<code>inicio.ts</code> (front)

¿Por qué solo el último mensaje?

Con el historial completo, los modelos pequeños a veces **repetían acciones antiguas** (por ejemplo, volvían a borrarlo todo). Enviar solo el último mensaje lo evita. El detalle está en `NOTAS.md`.

3.3 Iniciar sesión y permisos

#	Qué pasa	Dónde
1	<code>POST /api/auth/login</code> con email y contraseña. Se compara la contraseña con su hash bcrypt ; la contraseña real nunca se guarda.	<code>auth.service.ts</code>
2	Si es correcta, la API firma un JWT (un "pase" con el id del usuario, su rol y su empresa) y lo devuelve junto con el usuario y sus capacidades.	<code>auth.service.ts</code>
3	La web guarda el token y el interceptor lo añade a cada petición en la cabecera <code>Authorization: Bearer</code>	<code>auth.interceptor.ts</code>
4	En cada petición, el middleware <code>verificarToken</code> valida el JWT y comprueba que la empresa no esté suspendida .	<code>auth.middleware.ts</code>
5	Los guards de la web ocultan las pantallas que no te corresponden. Esto es solo por comodidad: la seguridad real está siempre en el backend.	<code>*.guard.ts</code> (front)

3.4 App móvil y gateway

#	Qué pasa	Dónde
1	Al iniciar sesión, la app llama a <code>/connected/gate</code> del gateway.	<code>mobile/.../core/auth.service.ts</code>
2	El gateway mira la IP y el país del móvil y los compara con las reglas del panel. Siempre responde con código 200 e indica en el cuerpo <code>ok: true/false</code> (el emulador de Android se bloqueaba con un 403).	<code>cloud-mid/src/index.ts</code> <code>filtros.ts</code>
3	Si puede entrar, hace el login contra la API de ATEKA y guarda el token con Preferences (el almacén de la app).	<code>auth.service.ts</code> (móvil)

#	Qué pasa	Dónde
4	El móvil envía un identificador de dispositivo. El panel muestra los dispositivos activos en los últimos 5 minutos.	<code>presencia.ts</code>
5	Al cerrar sesión, se llama a <code>/connected/salida</code> y el dispositivo desaparece del recuento.	<code>index.ts</code> (gateway)

En desarrollo y en producción el camino no es el mismo

Desarrollo: solo `/gate` y `/salida` van al gateway; la API se usa directamente (por un túnel al servidor).

Producción: **todo** pasa por el gateway, que reenvía `.../connected/api/...` a la API. Por eso el gateway necesita el mismo `JWT_SECRET` que ATEKA: verifica cada token antes de dejarlo pasar.

3.5 Automatizaciones con n8n

n8n es una herramienta para encadenar servicios (por ejemplo, "cuando llegue un mensaje de Telegram, pregúntale al asistente de ATEKA y responde").

- Cada usuario genera sus **claves API** en su página de **Perfil**. Puede tener hasta 5 activas. La clave completa se muestra **una sola vez**; la base de datos solo guarda su hash SHA-256, y se pueden revocar cuando se quiera.
- n8n envía la clave en la cabecera `X-API-Key` (no usa JWT). La petición actúa **como el usuario dueño de la clave**, con sus mismos permisos.
- `POST /api/n8n/facturas`: sube un fichero por la misma vía que la web, así que se indexa y se autoescanea igual.
- `POST /api/n8n/chat`: habla con el asistente. Si n8n envía `chat_id/user_id` (de Telegram), se devuelven tal cual para que n8n sepa a quién responder.
- Para pruebas existe también una clave fija en el `.env` (`N8N_API_KEY + N8N_USER_EMAIL`).

4. Tecnologías

Aquí solo aparecen las piezas importantes. Las librerías auxiliares (las que casi nunca vas a tocar) están en el **Anexo B**.

4.1 Backend

Lenguaje y motor

Tecnología	Para qué sirve
TypeScript	El lenguaje del código (<code>src/</code>). Es JavaScript con tipos , que permite detectar muchos errores al compilar y no en producción. Node no entiende TypeScript directamente: se traduce a JavaScript en la carpeta <code>dist/</code> .
Node.js 22	El motor que ejecuta ese JavaScript en el servidor. La imagen de Docker es <code>node:22-alpine</code> . La API escucha en el puerto interno 3000.
ts-node + nodemon	Solo para desarrollo: <code>npm run dev</code> arranca directamente desde <code>src/</code> y se reinicia al guardar.

Servidor HTTP

Tecnología	Para qué sirve
Express 5	El framework web: abre el puerto y define las rutas. Monta <code>/health</code> y nueve áreas: <code>/api/auth</code> , <code>/api/archivos</code> , <code>/api/chat</code> , <code>/api/facturas</code> , <code>/api/plataforma</code> , <code>/api/equipo</code> , <code>/api/compartido</code> , <code>/api/n8n</code> y <code>/api/claves</code> .
cors	Decide qué webs pueden llamar a la API desde el navegador (<code>CORS_ORIGIN</code>). A la app móvil no le afecta porque usa CapacitorHttp.
helmet	Añade cabeceras HTTP de seguridad estándar.
morgan	Escribe en el log una línea por cada petición recibida.
express-rate-limit	Limita cuántas peticiones se aceptan: el login, por IP (10 cada 15 min), y el chat, el escaneo y la subida, por usuario. Solo se activa en producción.
multer	Recibe los ficheros subidos (hasta 50 MB) y filtra por tipo.
archiver	Empaqueta una carpeta en un <code>.zip</code> para descargarla de una vez.

Datos y ficheros

Tecnología	Para qué sirve
PostgreSQL 16	La base de datos. Guarda usuarios, empresas, fichas de archivos, carpetas, facturas y la cola de tareas. No guarda el fichero en sí.
pgvector	Extensión de Postgres que guarda vectores (listas de números que representan el significado de un texto) y permite buscar los más parecidos.
unaccent	Extensión de Postgres para buscar ignorando las tildes: "facturacion" encuentra "facturación".
TypeORM	Traduce las tablas de Postgres a clases de TypeScript (<code>entities/</code>) para no escribir SQL a mano. Las migraciones (<code>migrations/</code>) cambian el esquema de forma ordenada.

Tecnología	Para qué sirve
MinIO	Almacén de ficheros compatible con Amazon S3. Guarda PDFs, imágenes y avatares en el "bucket" <code>archivos</code> .

Seguridad y configuración

Tecnología	Para qué sirve
jsonwebtoken (JWT)	Tras un login correcto, la API firma un token que el cliente envía en cada petición (<code>Authorization: Bearer ...</code>). No se usan cookies de sesión.
bcrypt	Guarda el hash de la contraseña, nunca la contraseña en sí.
zod	Valida datos. Al arrancar comprueba que el <code>.env</code> esté completo y, si falta algo, la API no arranca. También valida el cuerpo de muchas peticiones.

Inteligencia artificial (Ollama)

Ollama es un servidor que ejecuta modelos de IA en local. No es una librería de npm: la API lo llama por HTTP en `OLLAMA_URL`.

Modelo	Variable	Para qué sirve
qwen2.5-coder:14b	<code>OLLAMA_MODEL</code>	El asistente del chat y el uso de herramientas ("function calling"). También extrae los datos de las facturas y traduce al español las descripciones de imágenes que llegan en inglés. En máquinas pequeñas se usa la versión 7b o 3b.
bge-m3	<code>OLLAMA_EMBED_MODEL</code>	Convierte texto en un vector de 1 024 números . La columna de la base de datos tiene ese tamaño, así que si se cambia de modelo hay que cambiar también la columna.
granite3.2-vision	<code>OLLAMA_CAPTION_MODEL</code>	Primera pasada sobre una imagen: si tiene texto, lo transcribe; si no, describe la foto (en español). Es opcional : al arrancar no se comprueba si está instalado.
deepseek-ocr	<code>OLLAMA_OCR_MODEL</code>	Segunda pasada, más lenta y precisa. Solo se usa si la imagen parece una factura con importes, para no equivocarse en los dígitos.

Lectura de documentos

Tecnología	Para qué sirve
pdf-parse	Saca el texto de un PDF "normal" (no escaneado).
mammoth	Saca el texto de un Word (.docx).
tesseract.js	OCR en la CPU, sin GPU. Es el último recurso si Ollama no saca texto útil. Lee español, catalán e inglés; los idiomas están incluidos en <code>tessdata/</code> para no descargarlos de internet.
sharp	Procesa imágenes: redimensiona avatares y convierte las fotos a PNG antes de enviarlas a Ollama (los modelos de visión fallan a menudo con WEBP).

Pruebas y despliegue

Tecnología	Para qué sirve
Jest + supertest	Tests automáticos (<code>npm test</code>). supertest lanza peticiones HTTP reales contra la API sobre una base de datos de pruebas (<code>clouddrive_test</code>).
Docker (Dockerfile en dos etapas)	Etapas 1: instala dependencias y compila. Etapa 2: copia solo <code>dist/</code> y <code>tessdata/</code> y ejecuta <code>node dist/index.js</code> . Así la imagen final es más ligera.
docker compose	Arranca juntos Postgres, MinIO, la API y la web. La API espera a que la base de datos y MinIO estén listos.

4.2 Frontend web

Tecnología	Para qué sirve
Angular 22	El framework de la interfaz. Cada pantalla es un componente standalone (autónomo, sin módulos). El estado se guarda en signals : variables que actualizan la pantalla solas cuando cambian.
Angular Router	Define las URLs: <code>/login</code> , <code>/inicio</code> (chat), <code>/archivos</code> , <code>/facturas</code> , <code>/papelera</code> , <code>/perfil</code> , <code>/equipo</code> (admin) y <code>/plataforma</code> (superadmin).
HttpClient + interceptor	HttpClient hace las peticiones a <code>/api/...</code> . El interceptor <code>auth.interceptor.ts</code> añade el token a todas.
Guards	Protegen las rutas: <code>authGuard</code> (hay que estar logueado), <code>adminGuard</code> , <code>superadminGuard</code> y <code>chatGuard</code> (hace falta la capacidad <code>chat</code>).
RxJS	Librería para trabajar con datos que llegan con el tiempo (respuestas HTTP, navegación). Angular la usa por dentro.
marked	Convierte las respuestas del asistente (en markdown: listas, negritas, tablas) en HTML.
SCSS + variables CSS	Estilos. El tema está en <code>styles.scss</code> (color principal <code>--green</code> , radios, colores de fondo). El modo oscuro se activa con la clase <code>body.dark</code> .
@angular/build (Vite)	<code>ng serve</code> levanta la web en el puerto 4200 y la recarga al guardar. Un proxy (<code>proxy.conf.json</code>) envía <code>/api</code> a <code>localhost:3000</code> . <code>ng build</code> genera la versión final en <code>dist/</code> .
nginx	En producción sirve la web ya compilada. Si una ruta no es un fichero, devuelve <code>index.html</code> , así <code>/archivos</code> no da 404 al recargar. Reenvía <code>/api/</code> a la API con 300 s de margen y admite subidas de hasta 50 MB.
Node.js 24 (solo al compilar)	La imagen de Docker usa <code>node:24-alpine</code> solo para compilar. En el contenedor final no queda Node, solo nginx.

4.3 App móvil

Tecnología	Para qué sirve
Angular 22 + Ionic 9	Angular es el "cerebro" (rutas, servicios) e Ionic da el aspecto de app nativa (pestañas, cabeceras, alertas, menús). A diferencia de la web, usa un NgModule por página (el esquema clásico de Ionic).

Tecnología	Para qué sirve
Capacitor 8	Mete la web compilada (<code>www/</code>) dentro de un proyecto Android (<code>android/</code>) y da acceso a funciones del teléfono mediante plugins.
CapacitorHttp	En Android, las peticiones las hace el sistema nativo y no el navegador interno, lo que evita los problemas de CORS. El token se añade a mano.
@capacitor/preferences	Guarda el token, el usuario, el id del dispositivo y el tema. En el PC acaba en <code>localStorage</code> .
@capacitor/filesystem + file-opener	Descargan un fichero a la caché del móvil y lo abren con otra app (visor de PDF...).
Otros plugins	<code>keyboard</code> (teclado del chat), <code>status-bar</code> , <code>share</code> (compartir), <code>haptics</code> (vibración) y <code>app</code> (ciclo de vida).
ionicons + marked	Iconos y el markdown del chat, igual que en la web.

4.4 Gateway y panel de control (cloud-mid)

Tecnología	Para qué sirve
Node.js ≥ 20 + Express 5	El servidor del gateway. Rutas: <code>/health</code> , <code>/connected/gate</code> , <code>/connected/salida</code> , <code>/connected/*</code> (reenvío a ATEKA) y <code>/controlPanel</code> (panel).
tsx	Arranque en desarrollo con recarga (<code>npm run dev</code>). En producción (Plesk) se usa <code>node dist/index.js</code> .
geoip-lite + ipaddr.js	geoip-lite averigua el país a partir de una IP. ipaddr.js entiende IPv4, IPv6 y rangos (CIDR) para las reglas por IP.
http-proxy-middleware	Reenvía <code>/connected/...</code> a la API de ATEKA (<code>ATEKA_API_URL</code>) quitando el prefijo <code>/connected</code> .
jsonwebtoken	Si la petición trae un token, se verifica con el mismo <code>JWT_SECRET</code> que ATEKA (<code>src/jwtAteka.ts</code>). Un token falso se corta aquí con 401 y nunca llega a ATEKA. Si el secreto no coincide, se avisa en el log, en <code>GET /health</code> y en el panel. El login del panel usa un secreto propio, para que sus tokens no valgan en ATEKA.
MariaDB 11 + mysql2	La base de datos propia del gateway (no es la de ATEKA): reglas, intentos bloqueados, dispositivos y admins del panel.
Panel (HTML/JS/CSS)	Páginas estáticas de login, panel y perfil. Tiene su propio login de administrador (con <code>bcryptjs</code> y un JWT distinto del de ATEKA, válido 7 días).

5. Estructura de carpetas

Qué hay en cada carpeta y para qué sirve cada fichero importante. Las rutas son relativas a `akx-cloud-project/` salvo en el gateway.

5.1 Raíz del repositorio

Elemento	Qué es
<code>backend/</code>	La API (apartado 5.2).
<code>frontend/</code>	La web (apartado 5.3).
<code>mobile/</code>	La app Android (apartado 5.4).
<code>docker-compose.yml</code>	Servicios de producción: <code>db</code> (Postgres), <code>minio</code> , <code>api</code> y <code>web</code> .
<code>docker-compose.override.yml</code>	Solo en local: añade Ollama y Adminer (un visor web de la base de datos). En el servidor se borra, porque allí Ollama va aparte con GPU.
<code>.env</code> / <code>.env.example</code>	Secretos y configuración. El <code>.env</code> no se sube a git ; el <code>.example</code> es la plantilla.
<code>CLAUDE.md</code> / <code>NOTAS.md</code>	Documentación interna: referencia rápida y explicación detallada de las decisiones (chat, OCR, RAG, bugs).
<code>Documentos/</code>	Manuales, fichas y facturas de ejemplo.

5.2 Backend

Carpetas de primer nivel

Carpeta	Qué es
<code>src/</code>	El código de la API. Aquí se programa.
<code>tests/</code>	Tests de Jest: auth, archivos, avatar, chat (detección, roles, confirmación), claves, compartido, dedup, equipo, heurísticas de facturas, límites, n8n, plataforma y reconciliación.
<code>tessdata/</code>	Idiomas de Tesseract (<code>spa</code> , <code>cat</code> , <code>eng</code>). Van dentro de la imagen de Docker.
<code>dist/</code>	<i>Generada.</i> El JavaScript compilado desde <code>src/</code> . Es lo que se ejecuta en producción. No se edita a mano: se sobrescribe en cada compilación.
<code>node_modules/</code>	<i>Generada.</i> Las librerías instaladas con <code>npm install</code> . No se sube a git ni se modifica.

Dentro de `src/`, una petición sigue siempre el mismo camino:

Recorrido de una petición

`app.ts` → **routes/** (qué URL) → **middlewares/** (¿tiene token? ¿el id es válido? ¿supera el límite?) → **controllers/** (lee la petición) → **services/** (la lógica: base de datos, MinIO, IA) → respuesta.

Si algo falla, el servicio lanza un `AppError` y `errorHandler` lo convierte en una respuesta JSON `{ error }`.

Arranque: src/index.ts y src/app.ts

Fichero	Qué hace
<code>index.ts</code>	Punto de entrada. Conecta la base de datos (y ejecuta las migraciones), crea el superadmin si no existe, prepara el bucket de MinIO, comprueba los modelos de Ollama, arranca el worker y el mantenimiento y abre el puerto. Al apagarse (SIGTERM) cierra todo ordenadamente.
<code>app.ts</code>	Configura Express: helmet, cors, morgan, lectura de JSON (hasta 10 MB, por los avatares), <code>GET /health</code> y las nueve áreas <code>/api</code> . Al final, la respuesta 404 y el <code>errorHandler</code> . Activa <code>trust proxy</code> porque va detrás de nginx.

src/config/: configuración y conexiones

Fichero	Qué hace
<code>env.ts</code>	Lee el <code>.env</code> de la raíz y lo valida con zod. Si falta algo obligatorio, la API no arranca. Todo el código usa <code>env.ALGO</code> en lugar de <code>process.env</code> .
<code>database.ts</code>	La única conexión a Postgres (TypeORM): registra entidades y migraciones.
<code>minio.ts</code>	Cliente de MinIO. <code>inicializarBucket()</code> crea el bucket si no existe.
<code>ollama.ts</code>	Al arrancar comprueba que los modelos de chat, embeddings y OCR estén descargados. Si falta alguno, avisa en el log pero no para el servidor.
<code>capacidades.ts</code>	La lista fija de capacidades de los roles (<code>facturas</code> , <code>busqueda</code> , <code>gestion_archivos</code> , <code>chat</code>). Se guardan como <code>text[]</code> , así que añadir una nueva no requiere migración.

src/routes/ + src/controllers/: las URLs

Van por parejas: el **router** dice qué URL existe y qué filtros pasa; el **controlador** lee la petición, llama al servicio y responde. Ninguno de los dos contiene SQL.

Área	Router / controlador	Quién la usa	Qué cubre
<code>/api/auth</code>	<code>auth.*</code>	Todos	Login (público) y ver/editar el perfil.
<code>/api/archivos</code>	<code>archivos.*</code>	Usuarios	El explorador personal: subir, listar, buscar, descargar, carpetas, papelera y descripción manual de imágenes. Es el controlador más grande.
<code>/api/compartido</code>	<code>compartido.*</code>	Usuarios / admin	Carpetas compartidas: usarlas (cualquiera con el rol) y gestionarlás en <code>/admin</code> (crear, renombrar, asignar roles).
<code>/api/chat</code>	<code>chat.*</code>	Usuarios con <code>chat</code>	Un único <code>POST</code> que delega en <code>chat.service</code> . Puede tardar, porque llama a la IA.
<code>/api/facturas</code>	<code>facturas.*</code>	Usuarios	Listado, detalle, edición manual, escaneo (en cola) y reclasificación venta/compra.
<code>/api/equipo</code>	<code>equipo.*</code>	Admin	Miembros, roles, capacidades y datos de la propia empresa (CIF).
<code>/api/plataforma</code>	<code>plataforma.*</code>	Superadmin	Empresas: alta con su primer admin, edición, suspensión y borrado.

Área	Router / controlador	Quién la usa	Qué cubre
/api/claves	claves.*	Usuarios	Listar, crear y revocar las claves API propias para n8n.
/api/n8n	n8n.*	n8n (con X-API-Key)	/facturas (subir un fichero) y /chat (hablar con el asistente).

src/middlewares/: filtros antes del controlador

Fichero	Qué hace
auth.middleware.ts	verificarToken valida el JWT, rellena req.usuario y rechaza con 401 si no es válido o con 403 si la empresa está suspendida. soloAdmin y soloSuperadmin restringen por rol.
n8n.middleware.ts	Autentica las peticiones de n8n con la cabecera X-API-Key en lugar del JWT.
validarUUID.middleware.ts	Comprueba que el :id de la URL sea un UUID válido (si no, 400).
limites.middleware.ts	Límites por usuario y minuto: 40 mensajes de chat, 30 escaneos y 120 subidas. También impide encolar más de 200 tareas pendientes por usuario (MAX_BACKLOG_USUARIO), para que nadie sature la GPU.
errorHandler.middleware.ts	El último filtro: convierte cualquier error en una respuesta JSON uniforme.

src/services/: la lógica de negocio

Aquí está todo lo importante. Si un permiso o un listado falla, casi siempre es aquí donde hay que mirar.

Fichero	Qué hace
auth.service.ts	Login (bcrypt + firma del JWT) y perfil (nombre, avatar, contraseña).
archivos.service.ts	El explorador personal: guardar en MinIO y Postgres, listar, renombrar, copiar, papelera y restaurar, zip de carpetas y leer el texto para el RAG.
carpetas.service.ts	Carpetas personales: crear, mover, copiar, vaciar y borrar con todo su contenido en una sola operación.
compartido.service.ts	Carpetas compartidas: quién puede entrar (empresa + roles), operaciones con archivos, mover y copiar entre personal y compartido, y el historial de actividad (quién hizo qué).
chat.service.ts	El cerebro del asistente: comprueba permisos, prueba los pre-flights y, si ninguno encaja, dialoga con el modelo y ejecuta herramientas.
chat.tools.ts	El catálogo de herramientas que el modelo puede pedir (buscar, mover, facturas...). Si el rol no tiene la capacidad, la herramienta no se ofrece ni se ejecuta.
chat.deteccion.ts	Reconoce frases con reglas (regex): qué pide el usuario y de qué periodo habla (meses, trimestres, semestres). Tiene tests.
chatPendientes.service.ts	Guarda las preguntas a medio responder (aclaraciones, confirmaciones del tipo "¿seguro?") con fecha de caducidad.
claves.service.ts	Crea, lista y revoca las claves API. Guarda solo el hash y compara en tiempo constante (así no se puede adivinar la clave midiendo tiempos).

Fichero	Qué hace
<code>extraccion.service.ts</code>	Del fichero al texto: PDF, DOCX y TXT, y la cascada OCR para imágenes (granite → deepseek → Tesseract). Traduce al español las descripciones en inglés.
<code>facturas.service.ts</code>	Escanea facturas con IA, corrige emisor y cliente, clasifica venta/compra, aprende el CIF de la empresa, permite la edición manual y calcula la analítica.
<code>rag.service.ts</code>	Trocea los textos, genera los embeddings con bge-m3 y hace la búsqueda por significado (combinada con búsqueda por texto).
<code>tareas.service.ts</code>	La cola y el worker: recoge tareas pendientes, las reintenta si fallan (hasta 3 veces, con espera creciente) y limita cuántas van a la vez hacia la GPU (1 por defecto).
<code>reconciliacion.service.ts</code>	Mantenimiento al arrancar y cada 24 h: borra ficheros de MinIO que ya no tienen ficha (con más de 1 h de antigüedad), avisa de fichas sin fichero, vacía la papelera antigua (si se activa) y borra el historial de compartido de más de 365 días.
<code>equipo.service.ts</code>	Para el admin: miembros, roles y capacidades de su empresa (<code>capacidadesDe</code>), datos de la empresa y archivos de un miembro.
<code>plataforma.service.ts</code>	Para el superadmin: empresas y su primer admin (en una transacción: o se crea todo o nada).
<code>seed.service.ts</code>	Crea el superadmin al arrancar a partir de <code>SUPERADMIN_EMAIL</code> y <code>SUPERADMIN_PASSWORD</code> si todavía no existe ninguno.

src/entities/: las tablas

Cada clase es una tabla de Postgres. Los bytes de los ficheros **no** están aquí: están en MinIO.

Entidad	Tabla y contenido
Empresa	<code>empresas</code> : cada empresa cliente (nombre, CIF, activa/suspendida).
Usuario	<code>usuarios</code> : email, nombre, avatar, hash de la contraseña, rol y empresa.
Rol	<code>roles</code> : roles con nombre libre y su lista de capacidades.
Archivo	<code>archivos</code> : la ficha de cada fichero (nombre, carpeta, tipo, tamaño, clave en MinIO, hash, texto extraído, estados de escaneo e indexado, papelera).
Carpeta	<code>carpetas</code> : carpetas personales creadas explícitamente (también las vacías).
CarpetaCompartida	<code>carpetas_compartidas</code> : carpetas compartidas y los roles que tienen acceso.
CarpetaCompartidaCarpeta	<code>carpeta_compartida_carpetas</code> : subcarpetas dentro de una carpeta compartida.
EventoCompartido	<code>eventos_compartido</code> : historial de actividad de cada carpeta compartida (auditoría).
Factura / LineaFactura	<code>facturas</code> y <code>lineas_factura</code> : datos extraídos (número, fecha, emisor, cliente, NIF, tipo venta/compra, moneda, importes) y sus líneas.
Tarea	<code>tareas</code> : la cola del worker (tipo, estado, intentos, prioridad, error).
ChatPendiente	<code>chat_pendientes</code> : preguntas del chat a la espera de respuesta.
ClaveApi	<code>claves_api</code> : claves de n8n (nombre, prefijo visible, hash, último uso, revocada).

Otras carpetas de src/

Carpeta	Qué contiene
migrations/	El historial de cambios del esquema de la base de datos (21 ficheros con marca de tiempo). Se aplican solas al arrancar. Regla de oro: nunca se edita una migración ya aplicada; se crea una nueva.
utils/	errors.ts (la clase <code>AppError</code>) y tiposArchivo.ts (la lista de tipos de fichero permitidos).
types/	express.d.ts: le dice a TypeScript que <code>req.usuario</code> existe. No tiene lógica.

5.3 Frontend web

Raíz de frontend/

Fichero / carpeta	Qué es
src/	El código de la web.
public/	Ficheros estáticos que se copian tal cual (favicon, imágenes).
angular.json	Configuración del proyecto Angular (cómo compilar y servir).
proxy.conf.json	En desarrollo, envía <code>/api</code> a <code>localhost:3000</code> .
nginx.conf	Configuración de nginx en producción (SPA + proxy <code>/api</code>).
Dockerfile	Compila con Node 24 y sirve con nginx.

src/

Fichero / carpeta	Qué es
main.ts	Punto de entrada: arranca la aplicación Angular.
index.html	La única página HTML. Angular pinta todo dentro de ella.
styles.scss	Estilos globales y el tema (colores, modo oscuro).
environments/	Configuración por entorno (desarrollo/producción).
app/app.routes.ts	La lista de rutas y qué guard protege cada una.
app/app.config.ts	Registra el router y HttpClient con el interceptor.
app/app.ts	El componente raíz.

src/app/core/: servicios compartidos por toda la web

Fichero	Qué hace
auth.service.ts	Login y logout, guarda el token y el usuario, sabe el rol y las capacidades (<code>esAdmin()</code> , <code>puedeChat()</code> ...) y refresca el perfil al arrancar.
auth.interceptor.ts	Añade el token a cada petición.
auth.guard.ts, admin.guard.ts, superadmin.guard.ts, chat.guard.ts	Protegen las rutas. Si no tienes permiso, te redirigen.

Fichero	Qué hace
archivos.service.ts, compartido.service.ts, facturas.service.ts, equipo.service.ts, plataforma.service.ts, claves.service.ts	Uno por área de la API: funciones que llaman a sus endpoints.
chat.service.ts	El historial del chat (en un signal y en <code>localStorage</code> con la clave <code>akx_chat</code>). Se borra al cambiar de usuario para que no se mezclen conversaciones.
theme.service.ts	Modo claro/oscuro, recordado en <code>localStorage</code> (<code>akx_tema</code>).
toast.service.ts	Los avisos emergentes ("Archivo subido", errores...).
models.ts	Los tipos de TypeScript de los datos que devuelve la API (Archivo, Usuario, Factura...).

src/app/pages/: una carpeta por pantalla

Cada pantalla tiene tres ficheros: `.ts` (lógica), `.html` (plantilla) y `.scss` (estilos).

Pantalla	Ruta	Qué hace
login/	/login	Formulario de acceso.
inicio/	/inicio	El chat con el asistente (respuestas en markdown, tablas paginadas, botones "Abrir").
archivos/	/archivos	El explorador, con el selector Personales / Compartido (ver abajo).
facturas/	/facturas	Tabla de facturas con pestañas (Todas / Ventas / Compras / Sin clasificar), paginación y un editor para corregir lo que la IA extrajo mal.
papelera/	/papelera	Archivos borrados: restaurar o eliminar definitivamente.
perfil/	/perfil	Nombre, avatar, contraseña y claves API para n8n .
equipo/	/equipo	Solo admin: miembros, roles, carpetas compartidas y CIF de la empresa.
plataforma/	/plataforma	Solo superadmin: empresas.

La carpeta `pages/archivos/` merece una explicación, porque es la más compleja:

Fichero	Qué hace
archivos.ts	La página: el selector Personales / Compartido y la elección de la carpeta compartida.
explorador.ts / .html	El componente explorador : árbol de carpetas, arrastrar y soltar (también varios a la vez), menú contextual, selección múltiple, columna Estado y paginación.
fuentes.ts	La interfaz FuenteArchivos : la "forma" que debe tener un origen de datos.
compartido.ts	Adaptador que hace que las carpetas compartidas cumplan esa interfaz.
rutas.util.ts	Funciones puras para trabajar con rutas de carpetas (<code>/a/b</code>).

Por qué existe fuente.ts

Gracias a `FuenteArchivos`, **el mismo explorador** sirve para "Mis archivos" y para cada carpeta compartida. Si arreglas un fallo del explorador, queda arreglado en los dos sitios.

src/app/layout/ y src/app/shared/

Fichero	Qué hace
<code>layout/shell.*</code>	El marco común: la barra de navegación y el hueco donde se pinta cada pantalla.
<code>shared/errores.ts</code>	<code>mensajeError()</code> : convierte un error HTTP en un texto legible.
<code>shared/file-size.pipe.ts</code>	Muestra los bytes de forma legible ("2,3 MB").
<code>shared/password-input.component.ts</code>	Campo de contraseña con el botón para verla u ocultarla.
<code>shared/tipos-archivo.ts</code>	Extensiones permitidas. Debe coincidir con la lista del backend.
<code>shared/toasts.component.*</code>	Pinta los avisos de <code>toast.service</code> .

5.4 App móvil**Raíz de mobile/**

Fichero / carpeta	Qué es
<code>src/</code>	El código de la app.
<code>www/</code>	<i>Generada</i> . La app compilada (<code>ng build</code>). Capacitor la copia al proyecto Android.
<code>android/</code>	<i>Generada por Capacitor</i> . El proyecto nativo que se abre con Android Studio. Casi no se toca: solo carga la web.
<code>capacitor.config.ts</code>	Identificador de la app (<code>es.ateka.cloud</code>), nombre y ajustes. Permite HTTP sin cifrar para desarrollo .
<code>scripts/android-live.js</code>	<code>npm run android:live</code> : lanza la app en el emulador con recarga en vivo (busca el Java de Android Studio).
<code>ionic.config.json</code> , <code>angular.json</code>	Configuración de Ionic y Angular.

src/app/

Cada pantalla tiene cuatro ficheros: `*.module.ts` y `*-routing.module.ts` (su módulo y sus rutas), `*.page.ts` (lógica) y `*.page.html` (plantilla).

Carpeta / fichero	Qué hace
<code>app-routing.module.ts</code>	Rutas principales: <code>/login</code> y <code>/tabs</code> .
<code>tabs/</code>	La barra de pestañas inferior: Chat, Archivos, Facturas, Papelera y Perfil.
<code>login/</code>	Pantalla de acceso (primero pasa por el gateway).
<code>home/</code>	La pantalla del chat (pestaña Chat).
<code>archivos/</code> , <code>facturas/</code> , <code>papelera/</code> , <code>perfil/</code>	Las demás pestañas, equivalentes a las de la web.

Carpeta / fichero	Qué hace
<code>core/auth.service.ts</code>	Gate del gateway, login, cierre de sesión (<code>/salida</code>) y guardado en Preferences.
<code>core/archivos.service.ts</code> , <code>chat.service.ts</code> , <code>facturas.service.ts</code>	Llamadas a la API de ATEKA (CapacitorHttp en Android).
<code>core/urls.ts</code>	El único sitio con direcciones: <code>atekaUrl()</code> y <code>gatewayUrl()</code> . Los servicios las piden aquí, y aquí se elige el valor de navegador o el de Android.
<code>core/dispositivo.ts</code>	Genera y recuerda un identificador único del dispositivo (para el panel).
<code>core/auth.guard.ts</code> , <code>theme.service.ts</code>	Protección de rutas y modo oscuro.
<code>shared/</code>	Componentes reutilizables (botón del avatar) y su módulo.

Fuera de `app/` están `theme/` (variables de color de Ionic), `global.scss`, `assets/` (imágenes) y `environments/`, que merece explicación aparte.

Dónde están las direcciones del servidor

En `src/environments/`, y en ningún otro sitio. `environment.ts` es el de **desarrollo**: `/gate` va al gateway y la API va por el túnel del PC. `environment.prod.ts` es el de **producción**: todo pasa por el gateway de Plesk, que reenvía a la API.

`ng build` usa el de producción por defecto, así que es el que acaba en el APK. Para cambiar una dirección se toca solo ese fichero, nunca los servicios.

5.5 Gateway (repositorio cloud-mid)

Fichero	Qué hace
<code>src/index.ts</code>	Arranque y rutas: <code>/health</code> , <code>/connected/gate</code> (¿puede entrar?), <code>/connected/salida</code> (cierre de sesión), reenvío de <code>/connected/*</code> a ATEKA y el panel en <code>/controlPanel</code> .
<code>src/jwtAteka.ts</code>	Todo lo relacionado con el token de ATEKA: verificarlo y avisar si el <code>JWT_SECRET</code> no es el mismo que el de ATEKA.
<code>src/filtros.ts</code>	Obtiene la IP real del cliente y su país, carga las reglas (con caché), decide si entra y registra los intentos bloqueados.
<code>src/paises.ts</code>	La lista de países (código ISO y nombre) para el panel.
<code>src/presencia.ts</code>	Registra los dispositivos activos (ventana de 5 minutos) y los quita al cerrar sesión.
<code>src/panel.ts</code>	La API del panel: login, perfil, estado, dispositivos, reglas por IP y por país, e intentos.
<code>src/db.ts</code>	Conexión a MariaDB. Crea las tablas si no existen: <code>reglas_ip</code> , <code>reglas_pais</code> , <code>admins</code> , <code>intentos</code> y <code>dispositivos</code> .
<code>src/seed.ts</code>	Crea el primer administrador del panel.
<code>public/controlPanel/</code>	La web del panel: <code>index.html</code> + <code>login.js</code> (acceso), <code>panel.html</code> + <code>panel.js</code> (reglas, intentos, dispositivos), <code>perfil.html</code> + <code>perfil.js</code> , <code>common.js</code> , <code>styles.css</code> y el logo.
<code>docker-compose.yml</code>	Solo para desarrollo: una MariaDB local en el puerto 3307. En Plesk se usa la MariaDB del propio servidor.

Fichero	Qué hace
<code>.env.example</code>	Variables: <code>PORT</code> , <code>ATEKA_API_URL</code> (la API a la que reenvía), <code>JWT_SECRET</code> (el mismo que ATEKA), <code>PANEL_JWT_SECRET</code> (opcional, el del panel), <code>PROXY_TIMEOUT_MS</code> (180 s por defecto, para que quepa el chat) y los datos de MariaDB.

6. La API: la conexión entre las apps y el servidor

La web, la app móvil y n8n **hacen lo mismo**: llamar a la API. Todas las rutas empiezan por `/api/` y devuelven JSON. Salvo el login, todas exigen el token (`Authorization: Bearer ...`).

6.1 Áreas de la API

Área	Quién puede usarla	Qué ofrece
<code>/api/auth</code>	Todos	<code>POST /login</code> (la única ruta pública) y ver o editar el perfil: nombre, avatar y contraseña.
<code>/api/archivos</code>	Usuario	El espacio personal: subir, listar (paginado), buscar por significado, descargar, carpetas, papelera, copiar, renombrar y describir imágenes a mano.
<code>/api/compartido</code>	Usuario con el rol / admin	Carpetas compartidas: listar las accesibles, trabajar dentro de una (subir, mover, copiar, borrar, zip, buscar) y, en <code>/admin</code> , crearlas y asignarles roles.
<code>/api/chat</code>	Usuario con capacidad <code>chat</code>	Un único <code>POST</code> con la conversación. Devuelve el texto de la respuesta y, si procede, tablas de facturas, archivos o carpetas y los archivos a los que se refiere.
<code>/api/facturas</code>	Usuario con capacidad <code>facturas</code>	Listado con filtros y paginación, detalle, edición manual, escaneo (se encola) y reclasificar venta/compra.
<code>/api/equipo</code>	Admin de la empresa	Miembros, roles y capacidades, datos de la empresa (CIF) y archivos de un miembro.
<code>/api/plataforma</code>	Superadmin	Empresas: alta con su primer administrador, edición, suspensión y borrado.
<code>/api/claves</code>	Usuario (sus claves)	Crear, listar y revocar sus claves API. El secreto se enseña una sola vez .
<code>/api/n8n</code>	n8n, con <code>X-API-Key</code>	<code>/facturas</code> (subir un fichero) y <code>/chat</code> . No usa token: va con la clave API y actúa como el usuario dueño de esa clave.

Además, `GET /health` es pública y sirve para comprobar que el servidor está vivo.

6.2 Cómo se identifica cada cliente

Cliente	Camino	Cómo se identifica
Web (navegador)	nginx entrega la web y reenvía <code>/api/</code> a la API.	Token JWT, añadido a cada petición por el interceptor.
App móvil (desarrollo)	<code>/gate</code> al gateway y el resto directo a la API (túnel).	El mismo token JWT, añadido a mano (CapacitorHttp).
App móvil (producción)	Todo por el gateway: <code>.../connected/api/...</code> , que lo reenvía a la API.	El mismo token. El gateway además lo verifica: por eso necesita el mismo <code>JWT_SECRET</code> que la API.

Cliente	Camino	Cómo se identifica
n8n	Directo a <code>/api/n8n/....</code>	Cabecera <code>X-API-Key</code> con la clave creada en Perfil.

Dónde está el detalle

Esta tabla es el mapa general. La lista completa de rutas, con sus parámetros y respuestas, está en `CLAUDE.md`, en la raíz del repositorio, que se mantiene junto al código.

7. Base de datos

Hay **dos bases de datos distintas**, que no se comunican entre sí:

- **PostgreSQL 16**, la de ATEKA: usuarios, archivos, facturas y todo lo demás.
- **MariaDB 11**, la del gateway: reglas de acceso y dispositivos. Vive en el servidor de Plesk.

7.1 PostgreSQL (ATEKA)

Dos extensiones amplían Postgres: **pgvector**, que guarda los vectores de la búsqueda por significado, y **unaccent**, que permite buscar ignorando las tildes.

Las tablas las crean las **migraciones** (`backend/src/migrations/`, 21 ficheros), que se aplican solas al arrancar. En el código, cada tabla es una clase de `src/entities/`.

Identidad y permisos

Tabla	Campos principales	Relaciones
<code>empresas</code>	<code>nombre</code> , <code>nif</code> (CIF, se auto-aprende al escanear facturas), <code>estado</code> (activa/suspendida), <code>creadoEn</code>	Es el inquilino : de ella cuelga todo lo demás.
<code>usuarios</code>	<code>email</code> (único), <code>nombre</code> , <code>avatar</code> , <code>passwordHash</code> , <code>rol</code> (superadmin/admin/miembro), <code>creadoEn</code>	<code>empresaId</code> → <code>empresas</code> (nulo solo para el superadmin). Borrar la empresa borra sus usuarios.
<code>roles</code>	<code>nombre</code> (libre), <code>capacidades</code> (lista de texto)	<code>empresaId</code> → <code>empresas</code> . Nombre único dentro de cada empresa.
<code>usuario_roles</code>	<code>usuarioId</code> , <code>rolId</code>	Tabla puente: un usuario puede tener varios roles y un rol, varios usuarios.
<code>claves_api</code>	<code>nombre</code> , <code>prefijo</code> (visible), <code>hash</code> (único), <code>ultimoUso</code> , <code>revocadaEn</code> (vacío = activa)	<code>usuarioId</code> → <code>usuarios</code> . Las claves de n8n.

Archivos y carpetas

Tabla	Campos principales	Relaciones
<code>archivos</code>	<code>nombre</code> , <code>carpeta</code> (ruta), <code>mimeType</code> , <code>tamanoBytes</code> , <code>claveMinio</code> , <code>hashSha256</code> , <code>textoExtraido</code> , <code>descripcionManual</code> , <code>estadoEscaneo</code> , <code>estadoIndexado</code> , <code>eliminadoEn</code>	<code>propietario</code> → <code>usuarios</code> . <code>carpetaCompartidaId</code> vacío = archivo personal; con valor = vive en una carpeta compartida. <code>claveMinio</code> es el enlace con el fichero real en MinIO.
<code>carpetas</code>	<code>ruta</code> (única por propietario), <code>creadoEn</code>	Guarda las carpetas personales creadas a mano, también las vacías .
<code>carpetas_compartidas</code>	<code>nombre</code> (único por empresa), <code>creadoEn</code>	<code>empresaId</code> → <code>empresas</code> . N:N con roles vía <code>carpeta_compartida_roles</code> : quien tenga uno de esos roles, entra.
<code>carpeta_compartida_carpetas</code>	<code>ruta</code> , <code>creadaEn</code>	Las subcarpetas dentro de una carpeta compartida (equivale a <code>carpetas</code>).

Tabla	Campos principales	Relaciones
eventos_compartido	usuarioNombre, accion (subir, mover, eliminar...), objeto, ruta, detalle, creadoEn	Historial de quién hizo qué en una carpeta compartida. Solo texto: los ficheros siguen en MinIO.
fragmentos	indice, texto, embedding (vector de 1 024 números)	archivoId → archivos. Los trozos de texto con los que funciona la búsqueda por significado.

Facturas, cola y chat

Tabla	Campos principales	Relaciones
facturas	numero, fecha, emisor, emisorNif, cliente, clienteNif, tipo (venta/compra/desconocido), moneda, subtotal, iva, total	propietario → usuarios y archivo → archivos (la factura sobrevive si se borra el fichero).
lineas_factura	descripcion, cantidad, precioUnit, total	Las líneas de una factura. Se borran con ella.
tareas	tipo (indexar/autoescanear), estado, prioridad, intentos/maxIntentos, disponibleEn, error	La cola del worker. Al estar en la base de datos, el trabajo pendiente sobrevive a un reinicio .
chat_pendientes	tipo (aclaración, valor, confirmación), payload, expiraEn	Una fila por usuario: la pregunta del asistente que está esperando respuesta.

Los ficheros no están en la base de datos

Postgres guarda la **ficha** de cada archivo; el fichero en sí está en MinIO, y `claveMinio` los une. Por eso hay un mantenimiento periódico que comprueba que no falte ninguno de los dos lados.

7.2 MariaDB (gateway cloud-mid)

Base de datos aparte, con sus propias tablas. Las crea el gateway al arrancar si no existen (`src/db.ts`).

Tabla	Campos principales	Para qué
reglas_ip	cidr, accion (allow/deny), comentario, creado_en	Bloquear una IP o un rango.
reglas_pais	iso (código de 2 letras), accion, comentario	Bloquear un país entero.
intentos	ip, pais, metodo, ruta, email, motivo, permitido, creado_en	Historial: bloqueos e inicios de sesión. Se purga por antigüedad.
dispositivos	device_id (clave), ip, pais, email, conectado_en, visto_en	Quién está conectado ahora: se considera activo si se le ha visto en los últimos 5 minutos.
admins	email (único), password_hash, nombre, avatar	Las cuentas del panel de control. No son usuarios de ATEKA.

8. Seguridad

Resumen de las medidas de protección, de fuera hacia dentro.

Riesgo	Cómo se protege
Acceso desde móviles no deseados	El gateway filtra por IP y país antes de que la petición llegue a ATEKA, y corta los tokens falsos.
Robo de contraseñas	Solo se guarda el hash bcrypt. El login está limitado a 10 intentos cada 15 minutos por IP (en producción).
Suplantación	Todas las rutas (salvo el login) exigen un JWT válido. Si la empresa está suspendida, se bloquea en cada petición, no solo al entrar.
Ver datos de otra empresa	Cada consulta filtra por propietario o empresa. Las carpetas compartidas comprueban empresa + roles en el origen y en el destino.
Saltarse permisos	Las capacidades se comprueban en el backend . La web solo oculta botones; aunque alguien escriba la URL a mano, la API lo rechaza.
IA que hace lo que no debe	Solo ve las herramientas que su rol permite. Las acciones masivas irreversibles piden confirmación. Solo se le envía el último mensaje.
Ficheros maliciosos	Lista blanca de tipos, comprobación de los primeros bytes reales y tamaño máximo de 50 MB.
Saturar el servidor	Límites por usuario (chat, escaneo, subida) y un máximo de 200 tareas en cola por usuario.
Claves API filtradas	Solo se guarda el hash, se pueden revocar, hay un máximo de 5 por usuario y se registra su último uso.
Peticiones mal formadas	Validación con zod e ids comprobados como UUID.
Cabeceras y origen	helmet (cabeceras seguras) y CORS configurable (CORS_ORIGIN : en producción, el dominio de la web).
Privacidad de documentos	La IA es local: los documentos no se envían a servicios externos.
Secretos	El .env no está en git y se valida al arrancar.

9. Puesta en marcha y despliegue

9.1 Requisitos del servidor

Recurso	Recomendación orientativa
GPU	Para <code>qwen2.5-coder:14b</code> se recomienda una tarjeta de unos 16 GB de VRAM (el modelo ocupa unos 9 GB, más los de embeddings y visión). Con 8 GB, usa la versión 7b o 3b.
RAM	16 GB o más.
Disco	Depende del volumen de documentos (MinIO) más unos 20 GB para los modelos de IA.
Software	Docker con Docker Compose. Si hay GPU, los drivers de NVIDIA para Ollama.

Las cifras son orientativas. Con `ollama ps` puedes ver cuánta memoria ocupan realmente los modelos cargados.

9.2 Puertos

Puerto	Servicio	Notas
80	Web (nginx)	<code>WEB_PORT_HOST</code>
3000	API	<code>API_PORT_HOST</code>
5433	PostgreSQL	Dentro de Docker es el 5432. <code>DB_PORT_HOST</code>
9000 / 9001	MinIO (API / consola web)	<code>MINIO_PORT_HOST</code> / <code>MINIO_CONSOLE_HOST</code>
8080	Adminer	Solo en local
11434	Ollama	
4200	<code>ng serve</code> (web o móvil)	Solo en desarrollo
8081 / 3307	Gateway / su MariaDB	Solo en local

9.3 Variables del .env (las principales)

Grupo	Variables	Notas
Base de datos	<code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_NAME</code>	
MinIO	<code>MINIO_USER</code> , <code>MINIO_PASSWORD</code> , <code>MINIO_BUCKET</code>	Bucket por defecto: <code>archivos</code> .
Seguridad	<code>JWT_SECRET</code> , <code>CORS_ORIGIN</code>	<code>JWT_SECRET</code> : el código exige 16 caracteres como mínimo; se recomiendan 32 o más .
Superadmin	<code>SUPERADMIN_EMAIL</code> , <code>SUPERADMIN_PASSWORD</code>	Solo se usan si todavía no hay ningún superadmin.
IA	<code>OLLAMA_URL</code> , <code>OLLAMA_MODEL</code> , <code>OLLAMA_EMBED_MODEL</code> , <code>OLLAMA_CAPTION_MODEL</code> , <code>OLLAMA_OCR_MODEL</code>	Para cambiar de modelo basta con editar el <code>.env</code> y ejecutar <code>docker compose up -d api</code> .
Worker	<code>WORKER_CONCURRENCIA</code> (1), <code>WORKER_MAX_INTENTOS</code> (3), <code>MAX_BACKLOG_USUARIO</code> (200)	Entre paréntesis, el valor por defecto.

Grupo	Variables	Notas
Mantenimiento	MANTENIMIENTO_INTERVAL_HORAS (24), RETENCION_PAPELERA_DIAS (0 = desactivado), RETENCION_LOGS_DIAS (365)	
n8n	N8N_API_KEY, N8N_USER_EMAIL	Opcionales, solo para pruebas.

9.4 Comandos habituales

Desarrollo (lo más rápido: la infraestructura en Docker y el código en tu máquina, con recarga al guardar):

```
# Solo la infraestructura
docker compose up -d db minio ollama adminer
docker compose stop api web

# Backend en localhost:3000
cd backend && npm run dev

# Web en localhost:4200
cd frontend && npm start
```

Primera vez: descargar los modelos de IA:

```
docker exec clouddrive-ollama ollama pull qwen2.5-coder:14b
docker exec clouddrive-ollama ollama pull bge-m3
docker exec clouddrive-ollama ollama pull deepseek-ocr
docker exec clouddrive-ollama ollama pull granite3.2-vision
```

Despliegue en el servidor:

```
git pull
docker compose build api web && docker compose up -d api web

# Si solo cambia el .env (sin reconstruir)
docker compose up -d api
```

Cuidado

`docker compose restart` **no** recarga ni el código ni el `.env`: solo reinicia. Para código: `build + up -d`. Para el `.env`: `up -d`.

En el servidor **no se edita nada a mano**: los cambios se hacen en local → commit → push → `git pull`.

10. Limitaciones conocidas y pendientes

10.1 Limitaciones de diseño

- **Modelos pequeños:** el 3b y el 7b fallan a menudo al usar herramientas y a veces confunden el emisor y el cliente de una factura. Por eso existen los pre-flights, las correcciones automáticas y el **editor manual** de la página Facturas.
- **Tipos de fichero:** PDF, DOCX, XLSX, TXT, CSV, JPEG, PNG y WEBP. Máximo 50 MB. Un fichero por petición (la web sube varios en paralelo).
- **Worker de uno en uno** (por defecto): con muchas subidas a la vez, el procesado tarda más. Se puede subir `WORKER_CONCURRENCIA` si la GPU lo permite.
- **Carpetas compartidas:** sus archivos no van a la papelera (el borrado es definitivo y afecta a todos los del rol) y sus facturas se indexan pero **no** entran en la analítica.
- **Búsqueda del chat:** solo busca en los archivos personales. Cada carpeta compartida tiene su propio buscador en el explorador.
- **Facturas "Sin clasificar":** si no se conoce el CIF de la empresa y el nombre no coincide, quedan como desconocidas hasta que se corrigen o se pulsa "Reclasificar".
- **Retención de papelera** desactivada por defecto (`RETENCION_PAPELERA_DIAS=0`): lo borrado no se purga solo.

10.2 Pendientes antes de publicar la app

La app de producción lo envía **todo** por el gateway de Plesk, que lo reenvía a la API. Para que funcione fuera de la red local falta:

- **Que Plesk pueda llegar al servidor de ATEKA.** El servidor está en la red local y solo es accesible por SSH, así que Plesk todavía no tiene camino hasta él. Las dos salidas son un **túnel SSH inverso** desde el servidor hacia Plesk (cifrado, sin abrir puertos) o redirigir un puerto del router permitiendo solo la IP de Plesk.
- ``ATEKA_API_URL`` en el `.env` del gateway tiene que apuntar a la API del servidor por ese camino. Mientras valga `http://localhost:3000` (el propio Plesk), la app de producción no podrá iniciar sesión.
- **El mismo ``JWT_SECRET``** en el `.env` de ATEKA y en el del gateway: el proxy verifica cada token. Si no coinciden, la app da 401 después del login. Se comprueba en `GET /health` del gateway (`jwt.coincideConAteka`) y el panel avisa en rojo.
- **Tiempos de espera y tamaño en el nginx de Plesk:** más de 120 segundos de espera (el chat puede tardar) y 50 MB de subida, o cortará esas peticiones.

Anexo A. Glosario

Término	Qué significa
API	El "mostrador" del backend: una lista de direcciones (endpoints) a las que la web o la app piden cosas y que responden con datos (JSON).
Endpoint	Una dirección concreta de la API, por ejemplo <code>POST /api/archivos/subir</code> .
Backend / Frontend	Backend: lo que corre en el servidor. Frontend: lo que ve el usuario en el navegador o el móvil.
Docker / contenedor	Un contenedor es una "caja" con un programa y todo lo que necesita. Docker Compose arranca varias cajas a la vez (base de datos, API, web...).
.env	Fichero con la configuración y los secretos (contraseñas, claves). No se sube a git.
JWT / token	Un "pase" firmado que la API entrega al hacer login. El cliente lo enseña en cada petición para demostrar quién es.
Hash	Huella irreversible de un dato. Sirve para comprobar una contraseña sin guardarla, o para saber si dos ficheros son idénticos (SHA-256).
ORM (TypeORM)	Una capa que permite trabajar con tablas como si fueran clases de código, sin escribir SQL a mano.
Entidad	La clase que representa una tabla (por ejemplo, <code>Archivo</code>).
Migración	Un fichero que cambia la estructura de la base de datos (crear una tabla, añadir una columna) de forma ordenada y repetible.
Middleware	Un filtro que se ejecuta antes del controlador: comprueba el token, los límites, etc.
Controlador / servicio	El controlador atiende la petición HTTP; el servicio contiene la lógica de negocio.
Bucket (MinIO / S3)	Un "cajón" donde se guardan ficheros en un almacén de objetos.
OCR	Reconocimiento de texto en imágenes (leer una factura fotografiada).
Modelo de lenguaje (LLM)	La IA que entiende y genera texto (en este proyecto, qwen2.5-coder).
Function calling / tool	La capacidad del modelo de pedir que se ejecute una función ("mover archivo X") en lugar de solo responder con texto.
Pre-flight	En este proyecto: una regla fija que reconoce una frase habitual y la resuelve sin llamar a la IA.
Embedding / vector	Una lista de números que representa el significado de un texto. Dos textos parecidos tienen vectores cercanos.
RAG	Buscar los fragmentos de documentos más relevantes (por significado) para usarlos al responder.
Cola de tareas / worker	Lista de trabajos pendientes y el proceso que los va haciendo en segundo plano.
Multi-tenant	Una sola instalación que sirve a varios clientes (empresas) con los datos separados.
Rol / capacidad (RBAC)	Control de acceso por roles: cada rol tiene unas capacidades y cada usuario, unos roles.
Deduplicación	No guardar dos veces el mismo fichero: si el contenido es idéntico, se reutiliza.

Término	Qué significa
Magic bytes	Los primeros bytes de un fichero, que indican su tipo real aunque se cambie la extensión.
CORS	Regla del navegador que impide que una web llame a una API de otro dominio salvo que la API lo permita.
Proxy inverso	Un servidor intermedio que recibe peticiones y las reenvía a otro (nginx → API, gateway → ATEKA).
Rate limit	Límite de peticiones por tiempo para evitar abusos.
Soft delete / papelera	Marcar algo como borrado sin eliminarlo, para poder restaurarlo.
SPA	Single Page Application: una web que carga una sola vez y cambia de pantalla sin recargar.
Componente standalone	Un componente de Angular que declara él mismo lo que necesita, sin un módulo aparte.
Signal	Variable de Angular que avisa a la pantalla cuando cambia su valor.
Guard / interceptor	Guard: decide si se puede entrar en una ruta. Interceptor: modifica todas las peticiones HTTP (por ejemplo, para añadir el token).
Capacitor / WebView	Capacitor empaqueta una web como app nativa, que se muestra dentro de un navegador interno (WebView).
Gateway	Punto de entrada intermedio que filtra o reenvía el tráfico.
n8n	Herramienta de automatización visual que conecta servicios (Telegram, correo...) con APIs.
Markdown	Formato de texto sencillo (<i>**negrita**</i> , listas) que se convierte en HTML.

Anexo B. Librerías auxiliares

Estas librerías son necesarias, pero casi nunca vas a trabajar con ellas directamente.

Librería	Dónde	Para qué
reflect-metadata	Backend	TypeORM la necesita para leer los decoradores (<code>@Entity</code> , <code>@Column...</code>).
pg	Backend	El driver que conecta Node con PostgreSQL. TypeORM lo usa por debajo.
minio (SDK)	Backend	El cliente de Node para subir, bajar y borrar objetos en MinIO.
dotenv	Backend / gateway	Carga las variables del <code>.env</code> .
cross-env	Backend	Pone <code>NODE_ENV=test</code> igual en Windows que en Linux.
ts-jest	Backend	Permite a Jest ejecutar tests escritos en TypeScript.
tslib	Web / móvil	Funciones de ayuda que TypeScript inserta en el código compilado.
@angular/common, forms, platform-browser...	Web / móvil	Paquetes oficiales de Angular (plantillas, formularios, arranque en el navegador).
@angular/cli, compiler-cli	Web / móvil	Los comandos <code>ng serve</code> , <code>ng build</code> y <code>ng test</code> .
prettier	Web	Formatea el código automáticamente (espacios, comillas).
eslint, angular-eslint, vitest	Móvil	Revisión de estilo y tests (en el móvil apenas se usan).
@capacitor/cli	Móvil	<code>npx cap sync android</code> copia <code>www/</code> al proyecto Android.
bcryptjs	Gateway	Hash de la contraseña del admin del panel.
@types/*	Todos	Tipos de TypeScript para librerías escritas en JavaScript.